Resumo Executivo — Pipeline Produtos & Categorias

Bronze → Silver → Gold | Controle por Manifesto JSON, Observabilidade (Log + DQ) e KPIs de Margem | Correção Crítica de Escrita Indevida no RAW

Projeto	Estágio em Engenharia de Dados & IA — MercaData S.A.
Squad / Dupla	Squad 2 — Real Time for Business / Dupla 1
Responsável técnico	Lucas Sousa Santos Oliveira (parceiro de dupla: Zaiden Emiliano Segundo Seleme)
Escopo de dados	ecommerce_produtos, ecommerce_categorias
PR	*refactor(pipeline): camada de controle por JSON, observabilidade (log + DQ), Gold append-only, KPIs de margem e correção crítica de escrita no RAW*
Status	Execução ponta a ponta validada (01 → 03 → 04 → 05), 9 tabelas Gold confirmadas em Delta e SQL Server; correção crítica de escrita indevida no RAW aplicada.

5 notebooks no pipeline	**9** tabelas Gold (6 com espelho _atual)	**5** achados técnicos corrigidos nesta auditoria	**7/7** regras oficiais + 1 regra condicional de margem

Resumo

Reestruturação do pipeline completo de Produtos e Categorias, pedida pelo gestor com base em feedback estratégico: controle de ingestão por manifesto JSON (sem depender de mover/renomear arquivo físico), observabilidade por execução (log_pipeline + log_data_quality), Gold append-only por ciclo_id com KPIs de margem condicionais ao preço de custo, e a correção de um achado crítico de arquitetura — escrita indevida no container RAW. Inclui ainda auditoria técnica completa com 5 achados corrigidos.

Por que isso importa pro negócio

Este pipeline é a fonte de verdade que vai alimentar o dashboard do cliente no Looker. A correção crítica no RAW protege a propriedade do dado do produtor — o pipeline nunca deveria decidir o que já foi lido escrevendo na própria origem. A observabilidade nova (log + DQ + alerta automático no Teams) reduz o tempo entre um dado ruim entrar na quarentena e alguém do time saber disso, sem precisar abrir notebook. E o Gold passando a reter histórico completo (em vez de overwrite) é o que permite, pela primeira vez, mostrar evolução de catálogo e margem ao longo do tempo — não só uma foto.

Notebooks e Responsabilidades

Notebook	Responsabilidade	Decisão de negócio
`00_config`	Credenciais, conectividade, utilitários; controle por manifesto JSON; logs de observabilidade; alerta automático no Teams.	Elimina a dependência de mover/renomear arquivo físico para saber o que já foi ingerido.
`01 (Extração)`	Lê a landing do ADLS Gen2; 'já ingerido' decidido por manifesto + Bronze.	Container RAW permanece 100% leitura — o pipeline nunca decide o que já leu escrevendo na origem.
`03 (Bronze)`	Ingestão append-only, idempotente; removida escrita indevida no RAW (achado crítico).	Corrige violação de propriedade do dado: o RAW pertence ao produtor, nunca deveria ser escrito aqui.
`04 (Silver)`	Valida as 7 regras oficiais + 1 regra condicional de margem; upsert.	Alerta de margem negativa/zero assim que o preço de custo existir, sem quebrar o pipeline antes disso.
`05 (Gold)`	9 tabelas de negócio em Delta, append por ciclo_id, 6 com espelho _atual no SQL Server.	Histórico nunca é sobrescrito; BI lê o estado atual sem precisar filtrar ciclo manualmente.

KPIs de Negócio Entregues (camada Gold)

KPI (tabela Gold)	Pergunta de negócio que responde
`gold_kpi_catalogo_resumo`	Qual a saúde do catálogo agora? (cobertura, preço médio, concentração, frescor) — já inclui margem média e cobertura de custo informado: preco_custo foi criada pelo gestor em 08/10 e o cálculo já está ativo.
`gold_kpi_categoria_performance`	Quais categorias concentram valor e quais precisam de ação (sem oferta ativa, preço inconsistente, margem negativa — diagnóstico MARGEM_NEGATIVA já em produção)?
`gold_kpi_categoria_raiz`	Quais departamentos têm o maior sortimento e valor exposto?
`gold_kpi_faixa_preco`	Como o catálogo se posiciona em faixa de preço, por departamento?
`gold_kpi_qualidade_dados`	Que regra está barrando produtos de entrar no catálogo, e com que frequência (sem a dupla contagem corrigida nesta auditoria)?
`gold_kpi_rejeitados_produtos (novo)`	Quais SKUs estão em quarentena agora, por qual motivo, e já existe versão válida deles na Silver?
`gold_hist_* (3 tabelas)`	O catálogo está crescendo, encolhendo ou mudando de perfil ciclo a ciclo? (histórico completo — Gold deixou de usar overwrite)

Cobertura das regras oficiais da planilha: 5 de produtos + 2 de categorias, 100% implementadas e rastreáveis até o código; mais 1 regra condicional de margem (preco_custo, criada pelo gestor em 08/10) — já em produção, com proteção de schema para quando a coluna ainda não existia em janelas anteriores (tabela completa na primeira célula do notebook 04).

Regras de Negócio Implementadas (Camada Silver)

Tabela	Tipo	Nº	Regra	Camada onde é aplicada / exposta
ecommerce_produtos	**Técnica**	1	SKU deve ter comprimento dentro do padrão gerado (len > 5 e < 60)	Validação — Silver
ecommerce_produtos	**Técnica**	2	preco_lista deve ser > 0 e < 5000	Validação — Silver
ecommerce_produtos	**Técnica**	3	is_ativo deve ser boolean (True/False), sem nulos	Validação — Silver
ecommerce_produtos	**Negócio**	4	KPI: novos SKUs no micro-lote atual (> 50 pode ser carga de teste)	KPI/alerta — Gold (gold_kpi_catalogo_resumo)
ecommerce_produtos	**Negócio**	5	Alertar produto ativo com preco_lista 0 ou negativo	Quarentena — Silver · KPI/alerta — Gold
ecommerce_produtos	**Condicional**	6	preco_custo deve ser não nulo e > 0 (só quando a coluna existir); alertar margem negativa ou zero	Validação condicional — Silver · KPI/alerta — Gold (margem)
ecommerce_categorias	**Técnica**	1	id_categoria e nome_categoria obrigatórios em cada lote	Validação — Silver
ecommerce_categorias	**Negócio**	2	Alertar mudança no número de categorias raiz entre lotes	KPI/alerta — Gold (gold_kpi_catalogo_resumo)

Fonte: planilha oficial "regras_estagio_engdados_squad2_.xlsx", aba "Squad 2" — reproduzida sem alteração de conteúdo.

Decisões de Arquitetura

Decisão de Arquitetura	Impacto Técnico (Código)	Impacto de Negócio
Controle por manifesto JSON em vez de Landing Zone física	Cada execução grava `controle/ingestao/<tabela>/<run_id>.json`; 'já ingerido' não depende mais de mover ou renomear arquivo.	Container RAW nunca é escrito pelo pipeline — elimina a causa raiz do achado crítico abaixo.
Observabilidade por execução (log_pipeline + log_data_quality)	Duas tabelas Delta por run_id: volume/status/tempo/erro, e um check por regra técnica e de negócio.	Qualquer número estranho no dashboard é rastreável até o run_id e a regra exata que falhou, sem precisar abrir notebook.
Idempotência em toda camada	Bronze: anti-join por arquivo_origem (manifesto gravado só após append confirmado). Silver: upsert por chave + janela. Gold: append de histórico por ciclo_id.	Reexecutar qualquer notebook, em qualquer camada, não duplica dado — seguro a reprocessamento por erro humano ou falha de job.
Quarentena auditável + alerta automático no Teams	Todo registro rejeitado carrega motivo_rejeicao; notificação via webhook (urllib, biblioteca padrão) a cada item novo em quarentena.	Time sabe de um dado rejeitado no mesmo ciclo em que ele aconteceu, sem precisar consultar a tabela.
Upsert manual (anti-join + overwrite) em vez de DeltaTable.merge()	A credencial OAuth por chamada não é honrada pelo MERGE nativo neste workspace Serverless.	Mesmo resultado de negócio de um MERGE (upsert completo), sem depender de infraestrutura ainda não disponível (Unity Catalog).
SQL Server evolui schema sozinho (mergeSchema no Delta + resync automático)	Quando o Delta ganha coluna nova (ex.: preco_custo), a tabela espelho no SQL Server é recriada/resincronizada automaticamente — sem DDL manual.	Coluna nova de negócio chega ao BI sem pedido de intervenção manual no banco relacional.
Tabelas _atual no lugar de views via pymssql	import pymssql travava o kernel inteiro (SIGABRT) por incompatibilidade binária com o runtime Serverless. Substituído por tabelas `_atual` regravadas por ciclo.	BI lê estado atual sem risco de o pipeline inteiro cair por causa de uma lib de infraestrutura.

Decisão de Arquitetura	Impacto Técnico (Código)	Impacto de Negócio
Delta como fonte de verdade; SQL Server é espelho sincronizado a partir dele	Falha no espelho não desfaz o Delta nem derruba o histórico já calculado.	A execução seguinte reconcilia — um KPI correto no Delta e temporariamente ausente no SQL Server é indisponibilidade, não erro de dado.

Achados Corrigidos Nesta Auditoria

#	Achado	Correção aplicada
1	**Escrita indevida no container RAW (crítico)**	Removida a função que movia/renomeava arquivos dentro do RAW — o container pertence ao produtor dos dados e nunca deveria ser escrito por este pipeline.
2	**Dupla contagem em gold_kpi_qualidade_dados**	A regra de negócio (SKU ativo com preço inválido) deixou de ser injetada como linha extra na tabela técnica; agora vive só no resumo executivo.
3	**Vazamento de coluna de partição (data_particao)**	Coluna técnica da Bronze que vazava para a Silver — removida.
4	**Fragilidade de tipo na hierarquia de categorias**	Inconsistência int/double na hierarquia — corrigida com cast explícito.
5	**Divergência de schema Delta ↔ SQL Server**	O Delta evolui sozinho via mergeSchema, mas o SQL Server não; corrigido com recriação/resync automático da tabela espelho quando necessário.

Validação Realizada

- Execução ponta a ponta (01 → 03 → 04 → 05) com status SUCESSO e as 9 tabelas confirmadas SQL Server OK.
- Checagem de consistência interna: `gold_hist_categoria_share` = 135 categorias × nº de ciclos, batendo exato — sem duplicação nem perda entre ciclos.
- Confirmado que o container RAW permanece intacto após reprocessamento (achado crítico corrigido).
- Auditoria completa das regras de negócio contra casos de borda (lote vazio, primeira carga, versão antiga, categoria sem produto, SKU órfão, JSON de controle legado).

Como Testar

1. Rodar `03` (Bronze), depois `04` (Silver), depois `05` (Gold), nessa ordem (00 e 01 são herdados via %run, não rodam sozinhos).
2. Conferir o manifesto JSON e os logs em `log_pipeline/log_data_quality` ao final de cada notebook.
3. Validar as 9 tabelas `gold_*` em Delta e no SQL Server (schema `dbo`), incluindo as 6 tabelas `_atual`.

Pendências Conhecidas (não bloqueiam este PR)

- Limites de negócio da Gold (corte de classe de valor, dispersão de preço) são constantes parametrizadas no topo do notebook 05 — calibrar com o time de negócio antes da apresentação.
- preco_custo já chegou (08/10) e os KPIs de margem já calculam; a condição no código fica como proteção contra janelas antigas sem a coluna, não como bloqueio — cobertura ainda parcial enquanto o gestor completa o rollout em todo o catálogo.
- OPTIMIZE ZORDER desabilitado até a migração para Unity Catalog destravar MERGE/OPTIMIZE nativos.

■ Reconciliação de SKUs órfãos (hoje zero no catálogo): documentada no modelo de dados para quem for montar o Looker — `gold_kpi_catalogo_resumo.total_skus` inclui órfãos, as tabelas por categoria não. Sem ação de código necessária enquanto o valor for zero.